

Manual de Desarrollo

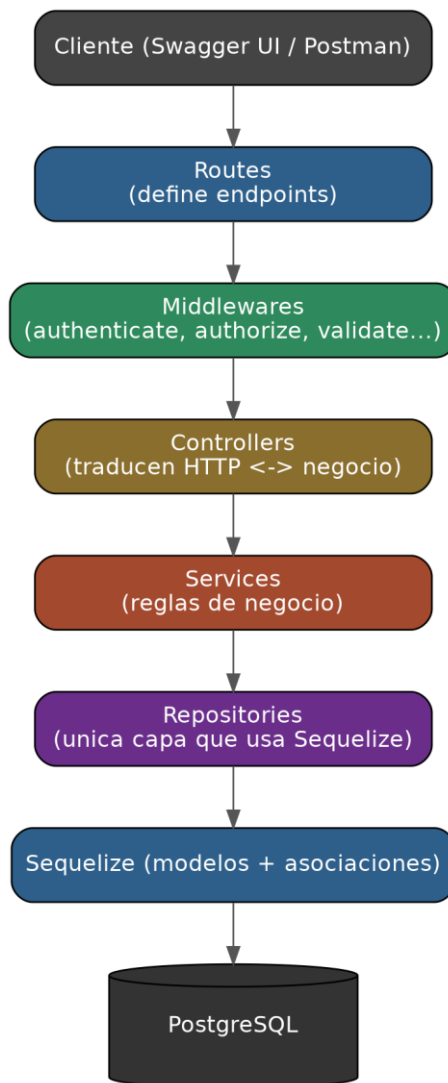
RiwiMediCare Plus API — Documentación técnica

1. Stack tecnológico

- Node.js 20, Express 5, TypeScript estricto
- PostgreSQL 15, Sequelize 6 (modelos tipados con InferAttributes/InferCreationAttributes)
- Autenticación: JWT (jsonwebtoken) + bcryptjs para hash de contraseñas
- Documentación: Swagger/OpenAPI (swagger-jsdoc + swagger-ui-express)
- Carga de archivos: Multer
- Contenedores: Docker + Docker Compose
- Control de versiones: Git con GitFlow (main / develop / feature/*) y Conventional Commits

2. Arquitectura por capas

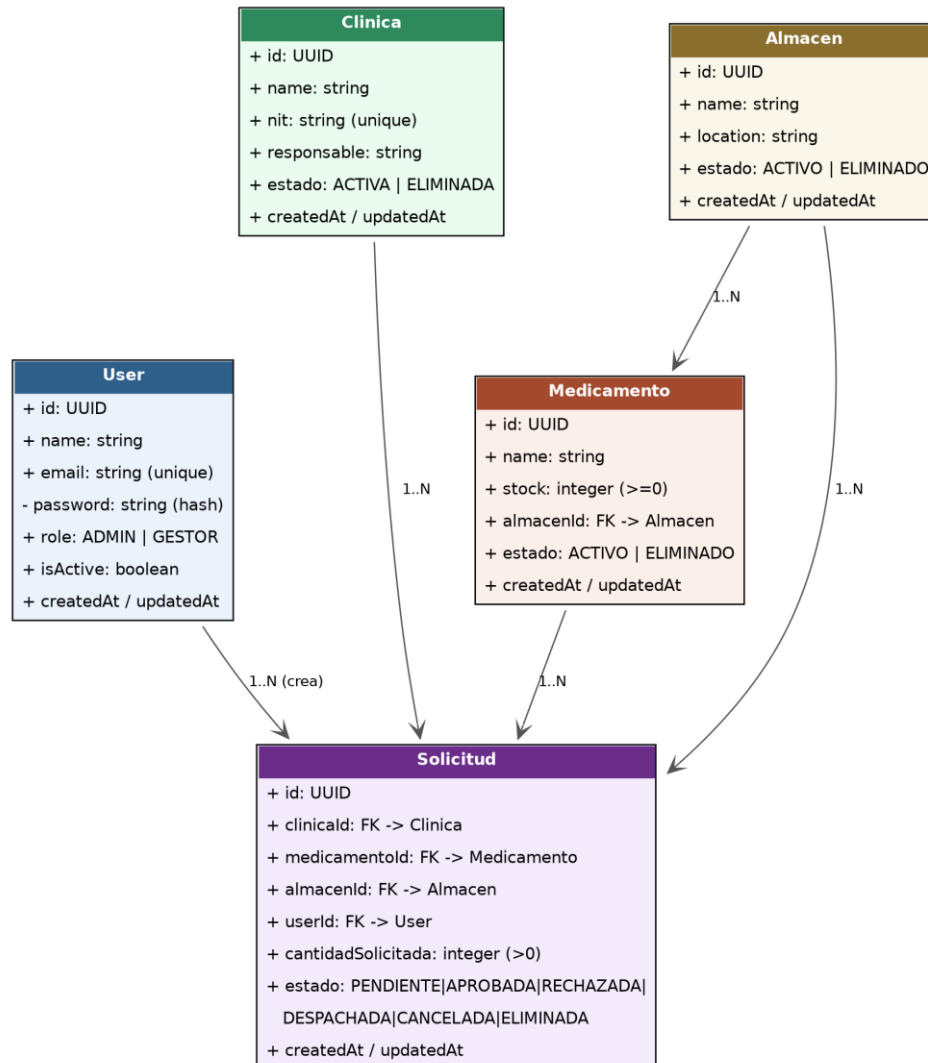
El proyecto sigue una arquitectura estricta por capas, donde cada capa solo se comunica con la inmediatamente inferior:



Capa	Responsabilidad	No debe hacer
------	-----------------	---------------

routes	Define endpoints y los documenta con JSDoc/OpenAPI	Contener lógica de negocio
middlewares	Autenticación, autorización, validación de entrada, errores	Acceder directamente a la base de datos
controllers	Traduce HTTP <-> negocio; delgados	Contener reglas de negocio
services	Reglas de negocio (stock, NIT único, estados válidos)	Hablar con Sequelize directamente
repositories	Única capa que usa Sequelize; consultas y persistencia	Contener reglas de negocio
models	Entidades y asociaciones Sequelize	—

3. Modelo de datos (diagrama UML)



Relaciones: Almacen 1:N Medicamento · Clinica 1:N Solicitud · Medicamento 1:N Solicitud · Almacen 1:N Solicitud · User 1:N Solicitud (quien la crea).

Borrado lógico: Clinica, Almacen, Medicamento y Solicitud no se eliminan físicamente; su campo "estado" cambia a un valor de tipo ELIMINADA/ELIMINADO.

4. Estructura de carpetas

```
src/      config/      -> conexión a BD y variables de entorno  models/      -> entidades
Sequelize y asociaciones (index.ts)  dto/      -> interfaces de entrada por endpoint
repositories/ -> única capa que toca Sequelize  services/      -> reglas de negocio
controllers/ -> handlers HTTP, delgados  routes/      -> definición de endpoints +
swagger JSDoc  middlewares/ -> authenticate, authorize, validaciones, errores  errors/
-> AppError  docs/      -> configuración de swagger  utils/      -> jwt, password,
paginación  types/      -> extensión de Express Request (req.user)
```

5. Manejo de roles y autorización

El rol vive en el modelo User (enum Role: ADMIN | GESTOR) y viaja dentro del JWT emitido en el login. El middleware "authenticate" valida el token y adjunta el usuario a req.user; el middleware "authorize(...roles)" compara req.user.role contra los roles permitidos para ese endpoint y responde 401/403 si no corresponde.

El registro (POST /api/auth/register) es la única ruta pública: el propio usuario elige su rol al registrarse, sin validación por token, tal como lo exige el enunciado de la prueba.

6. Endpoints principales

Método	Ruta	Acceso
POST	/api/auth/register	Público
POST	/api/auth/login	Público
GET	/api/auth/me	JWT
GET/POST/PATCH/DELETE	/api/clinicas	GET: JWT · escritura: ADMIN
GET/POST/PATCH/DELETE	/api/almacenes	GET: JWT · escritura: ADMIN
GET/POST/PATCH/DELETE	/api/medicamentos	GET: JWT · escritura: ADMIN
POST	/api/solicitudes	ADMIN, GESTOR
PUT	/api/solicitudes/:id/estado	ADMIN, GESTOR
GET	/api/solicitudes/activas	JWT
GET	/api/solicitudes/historial	JWT
GET	/api/clinicas/:id/solicitudes	JWT
DELETE	/api/solicitudes/:id	ADMIN
POST	/api/seed/upload	ADMIN (multipart)

7. Endpoint de seed con Multer

POST /api/seed/upload usa multer con almacenamiento en memoria (memoryStorage) y un límite de 5MB. Recibe un archivo .json en el campo "file", lo parsea, y el service de seed inserta en orden: usuarios, clínicas, almacenes y medicamentos, resolviendo referencias por nombre (por ejemplo, un medicamento puede indicar almacenName en vez de un ID).

8. Backup de base de datos

```
# Local pg_dump -h localhost -p 5432 -U postgres desempeño > backup.sql # Docker docker
compose exec db pg_dump -U postgres exam_db > backup.sql # Restaurar psql -h localhost -U
postgres -d desempeño < backup.sql
```

9. GitFlow

- main: versión estable/entregable.
- develop: integración de todas las features.
- feature/*: una rama por bloque funcional (clinicas, almacenes, medicamentos, solicitudes, seed, docs/docker), mergeada a develop al terminar.
- Conventional Commits en todo el historial (feat:, fix:, docs:, refactor:).

10. Ejecución del proyecto

```
npm install cp .env.example .env npm run dev # http://localhost:3002/api npm run  
typecheck npm run build npm run docker:up # http://localhost:3003/api/docs/
```